



AUGUST 30 | IMC-21

Artificial Intelligence Methods for Microscopy Analysis and

University of Liverpool, UK

Rama Vasudevan¹ and Gerd Duscher²

¹Center for Nanophase Materials Sciences, Oak Ridge National Laboratory

²Department of Materials Science and Engineering, University of Tennessee



U.S. DEPARTMENT OF
ENERGY

ORNL IS MANAGED BY UT-BATTELLE LLC
FOR THE US DEPARTMENT OF ENERGY

Agenda

09:00- 10:00: Introduction to AI/ML in microscopy (Rama)
10:30 - 11:00 Morning Break
11:00 - 11:30 Pycroscopy, sidpy and SciFiReaders (Gerd)
11:30 - 12:00 Tutorial session (Gerd)- Remote Aberration Correction
12:00 -13:00 Lunch break
13:00 - 14:00 Knowledge extraction from images (Rama)
14:00 - 14:30 Autonomous electron microscopy demonstrations (Gerd)
14:30 - 15:00 Autonomous scanning probe microscopy (Rama)
(demonstrations + presentation)
15:00- 15:30 Afternoon Break
15:30 - 16:30 AI assisted data acquisition in scanning probe and electron
microscopy (digital twin demos)
16:30 - 17:00 Concluding remarks/discussion

What you need today:

- Laptop
- Attention
- Curiosity

All source material is
available at the
pycroscopy repo
linked below.

<https://github.com/pycroscopy/IMC21-Workshop>

Why this workshop?

- You are here, so it must be important....
- More seriously, machine/statistical learning is about finding correlations and relationships in datasets
- With large data sizes and more data dimensions, it is increasingly more difficult to analyze data using traditional methods. Even visualizing trends in multidimensional datasets can be a formidable challenge
- Computer science has developed tools to both analyze and visualize data, and modern LLMs provide new capabilities to inject prior knowledge and control workflows in ways that were previously limited to numerical algorithms

What *is* Machine learning?

- There are lots of definitions, but the main essence of machine learning is for the system to learn relationships from data, enabling generalizations to new situations. This is as opposed to being specifically programmed.
- Machine learning methods can be used for the following tasks:
 - Classification (can I group my observations in some manner?)
 - Regression (can I fit the data, even if the function cannot be expressed analytically?)
 - Dimensionality Reduction (can we express the data in a shorter form?)

Two types of ML

Supervised

“Give me some examples”

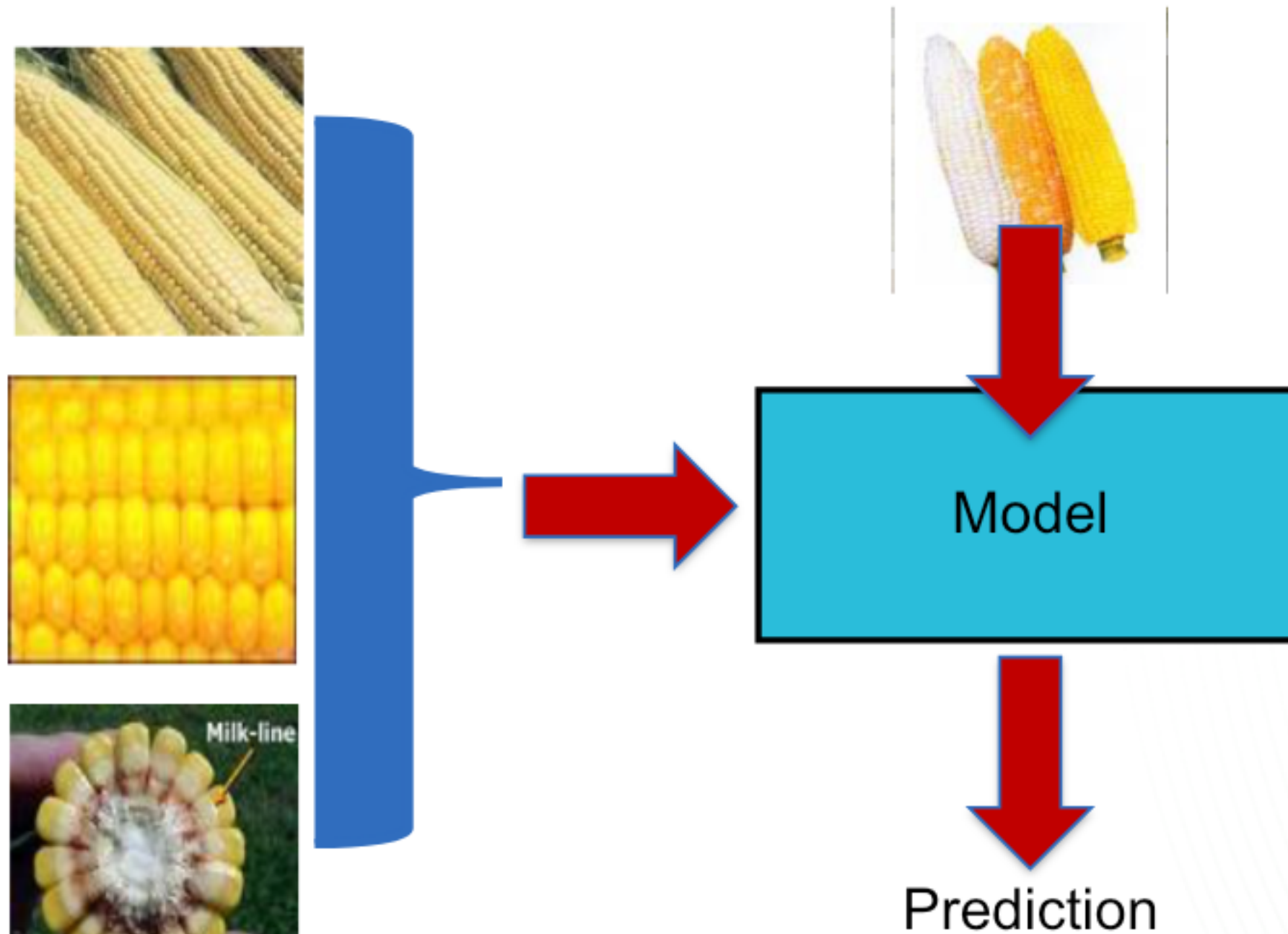
Unsupervised

“I don’t need no examples”

**In both cases: Machine learning models learn
from the data at hand**

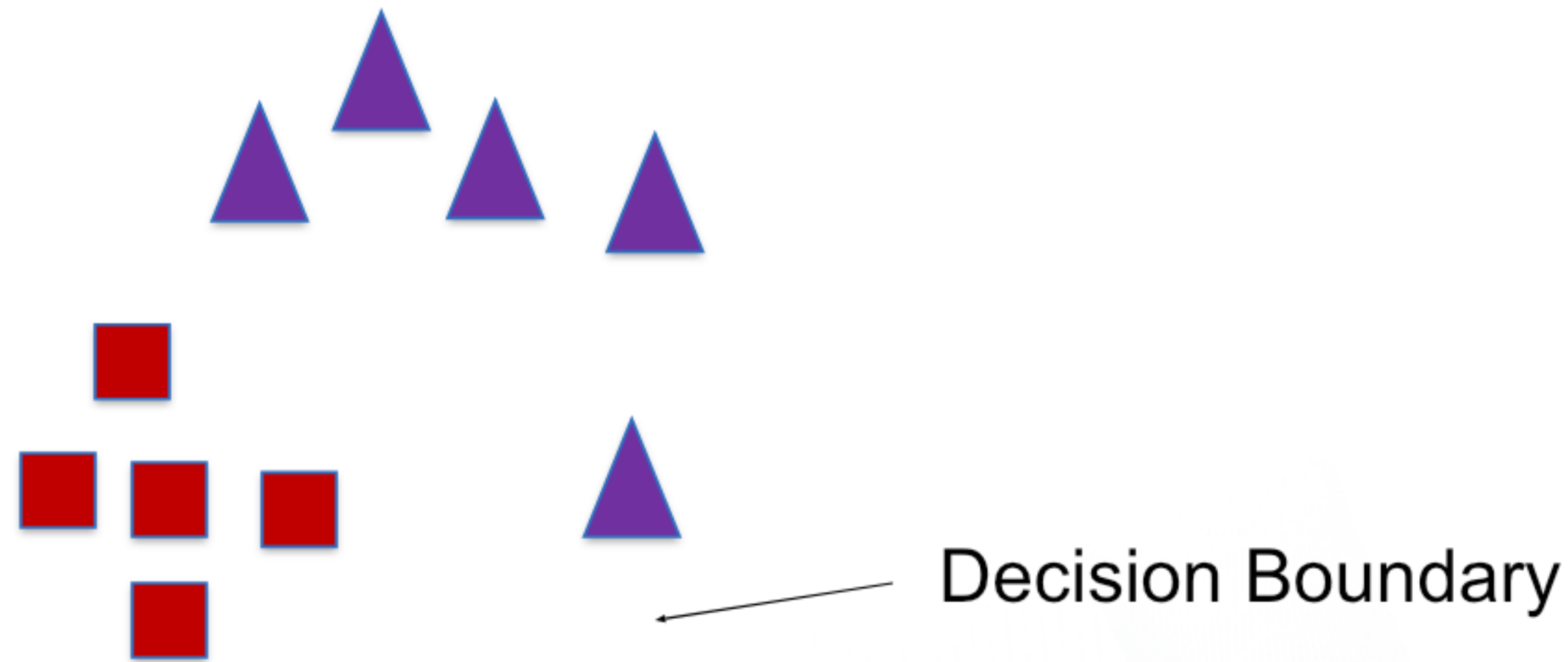
Supervised Classification

- Supervised classifiers can automatically classify data once the model has been trained on a 'training set'.



Supervised Classification

- Support Vector Machines are one common method, and is conceptually easy to understand.

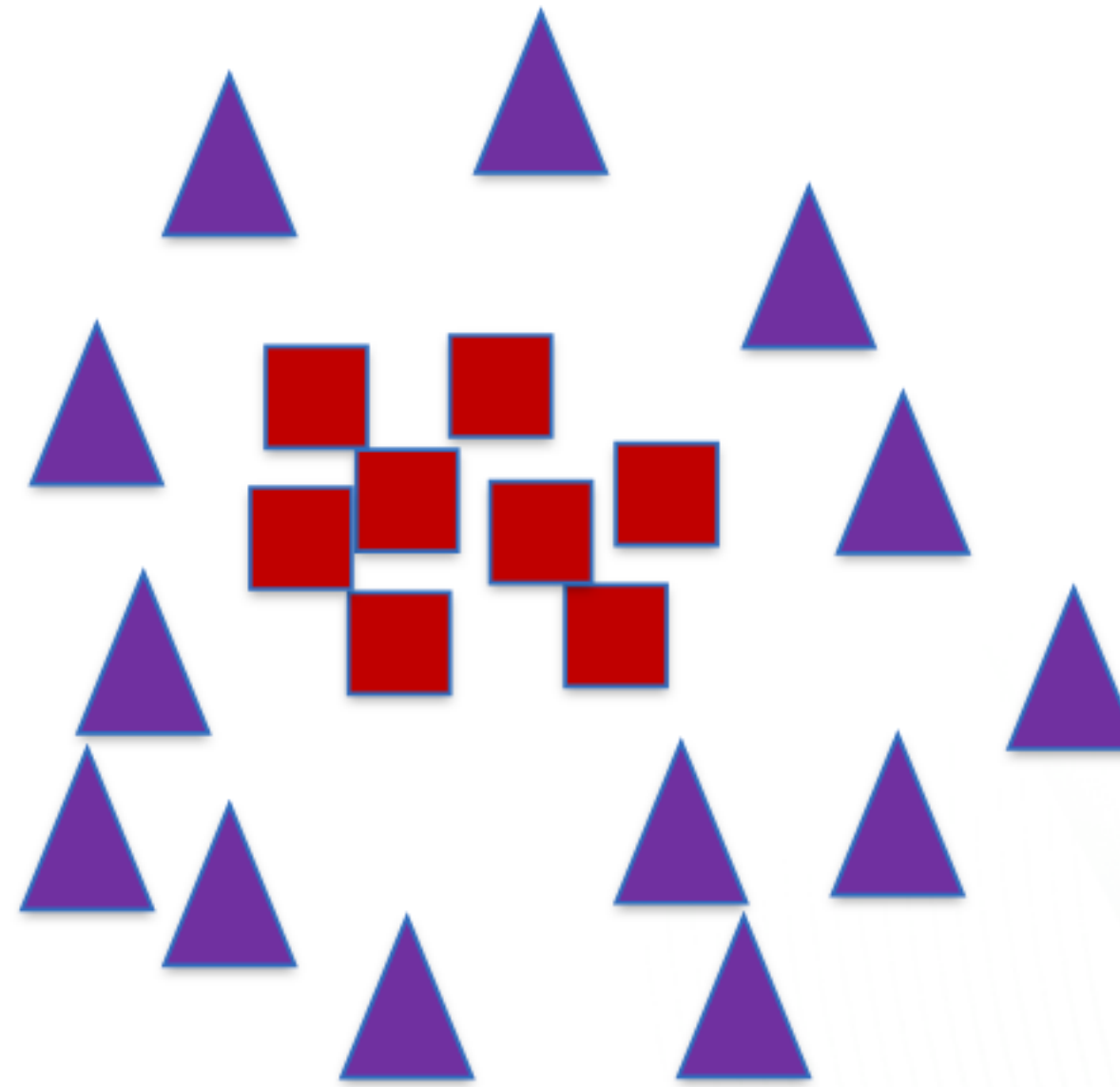


SVM are linear classifiers that fits lines (planes/hyperplanes in higher dimensions), such that the margin (separation between the line and the nearest training point) is maximized.

Supervised Classification

- But what to do when it cannot be linearly separated? E.g.,

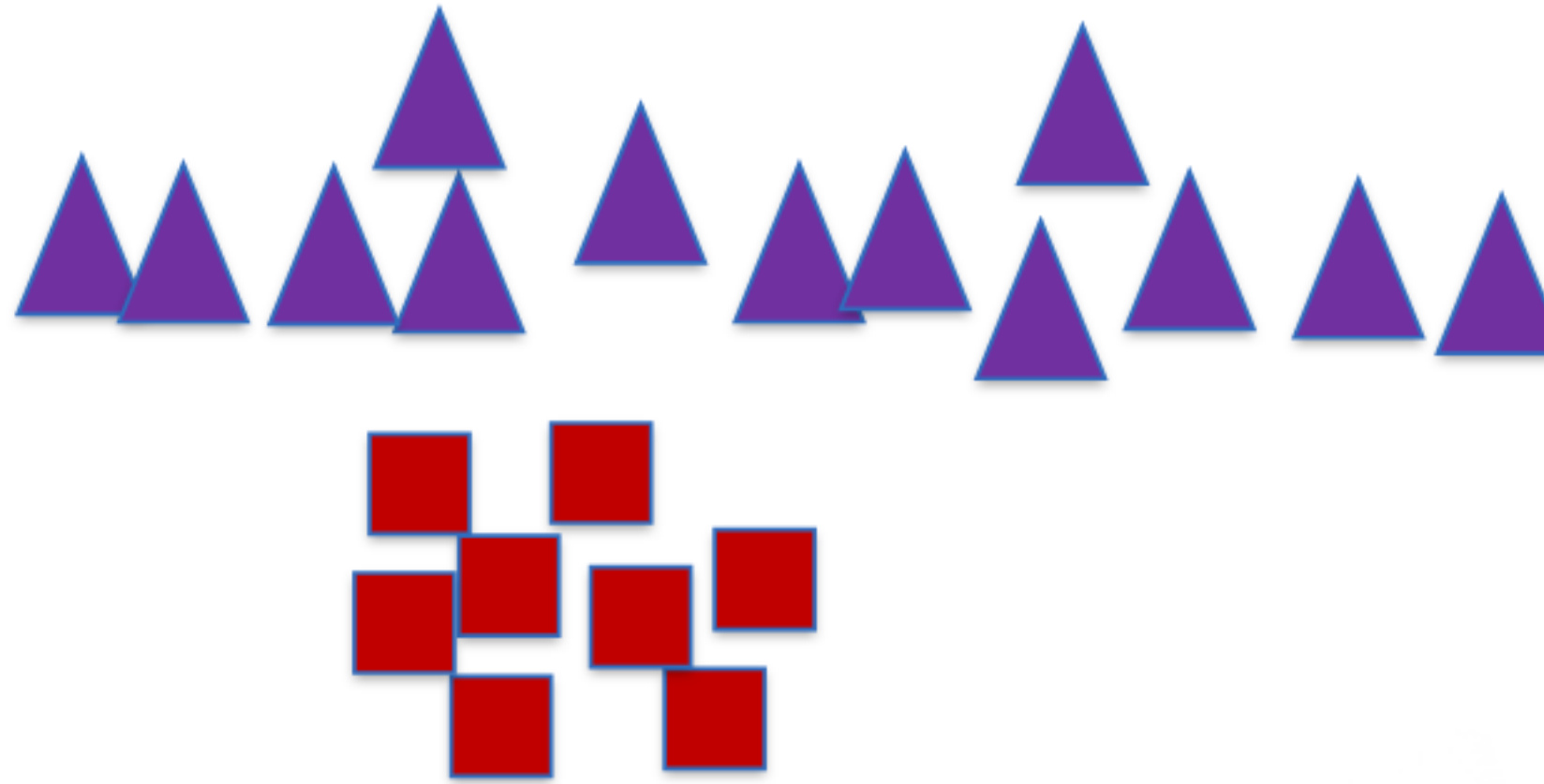
We cannot fit a line that will separate these two classes in this representation



Supervised Classification

- Solution: Transform to a space where they can be separated.

Apply a
transformation
 $Z = x^2 + y^2$



Unsupervised Classification

- In many cases, we do not have labeled examples. In this case, we can turn towards unsupervised methods
- The most common method is k-means clustering, but there are numerous others
- In the interests of time, we will simply explore some of the other clustering methods in a notebook

K-Means Clustering

- SVD, NMF and ICA are decompositions. But sometimes, we don't want to decompose our signal, but just group them into 'alike' sets.
- This is termed 'clustering'. The easiest and most widely used method is the k-means algorithm

K-means Clustering algorithm, to separate data $(x_1, x_2, \dots, x_n)$ into k clusters

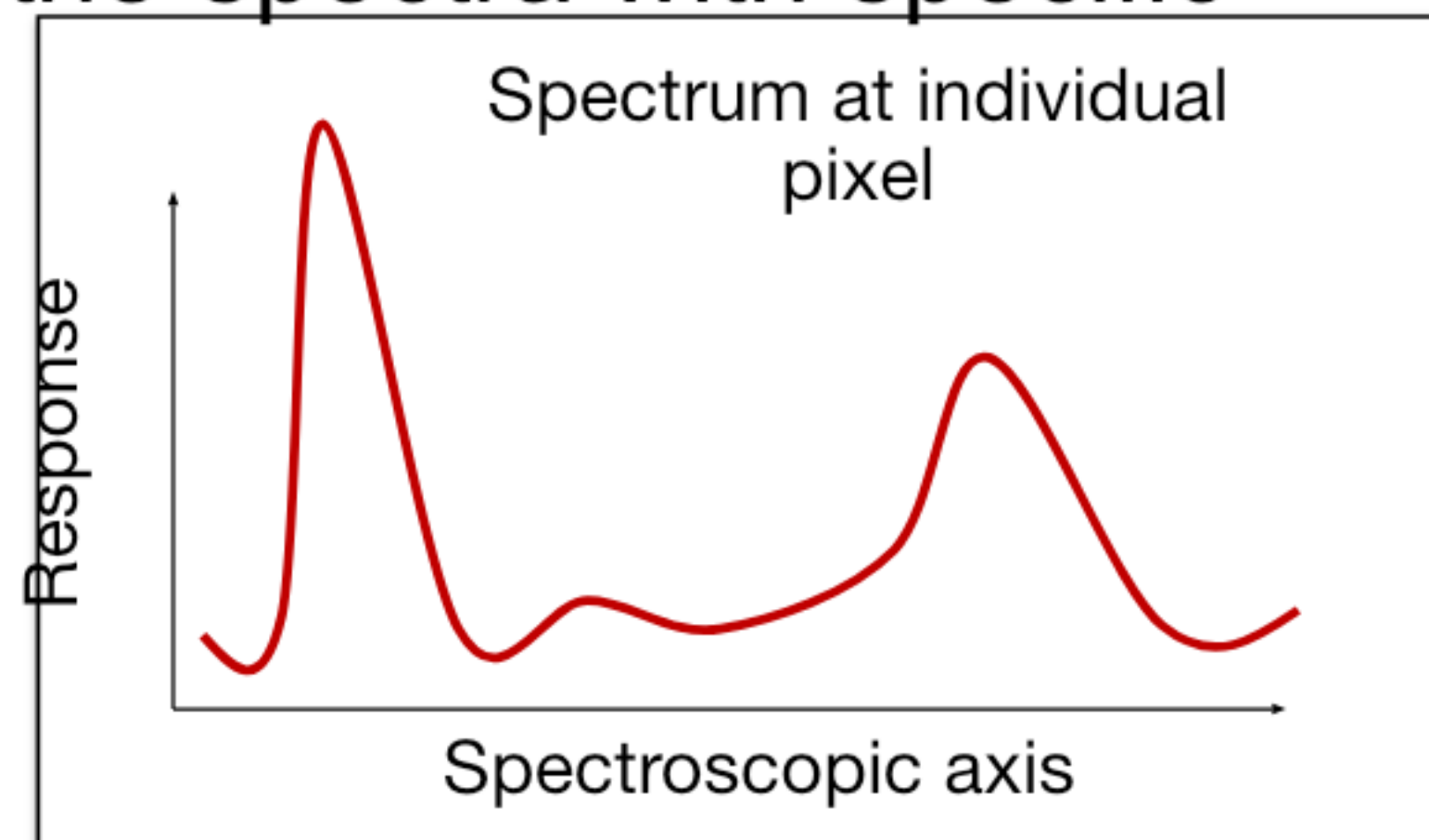
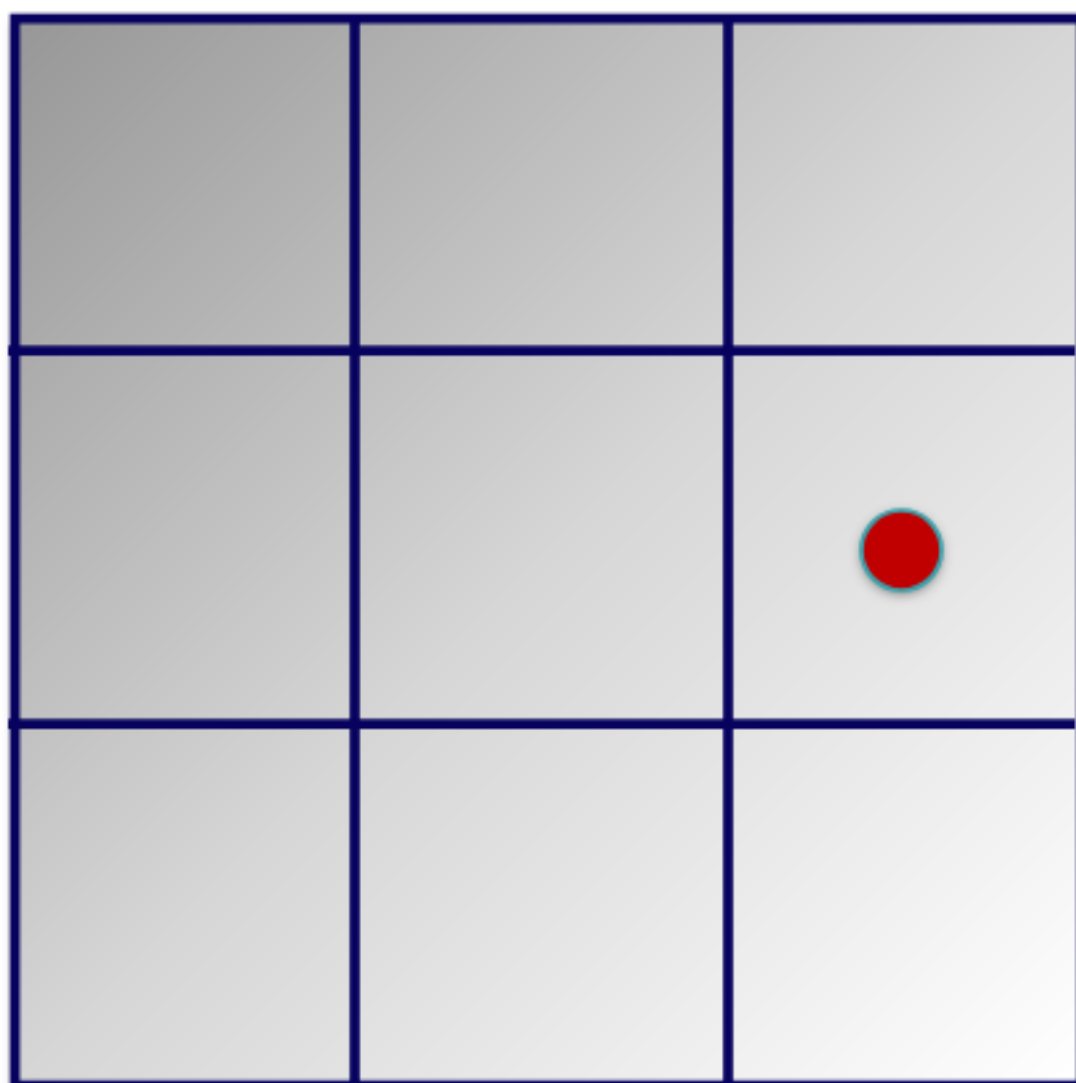
$$\arg \min_{\mathbf{S}} \sum_{i=1}^k \sum_{\mathbf{x} \in S_i} \|\mathbf{x} - \boldsymbol{\mu}_i\|^2 \quad \text{where } \boldsymbol{\mu}_i \text{ is the mean of points in } S_i$$

(Determine $\mathbf{S} = \{S_1, S_2, \dots, S_k\}$, such that within cluster sum of squares is minimized)

Spectral Data

- Many imaging modalities have spectroscopy components
- By capturing spectra across an image, it allows us to correlate features of the spectra with specific regions of the sample

Image (MxN pixels)



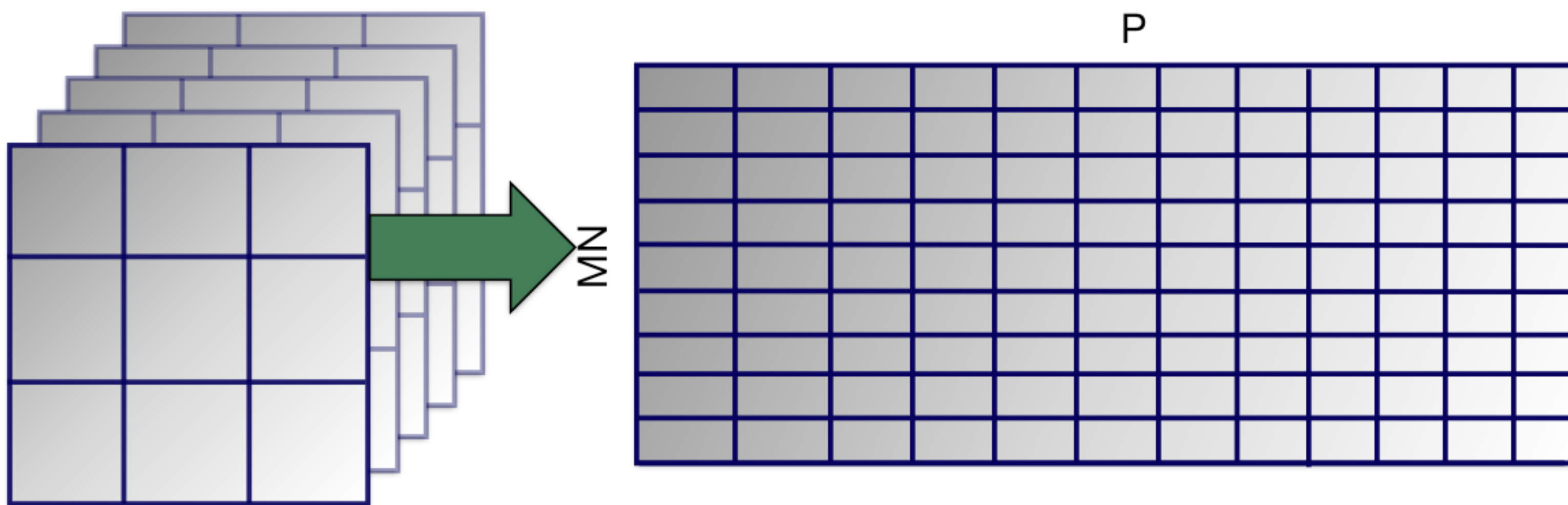
M (rows) x N (columns) x P (spectral points)

Spectral Data

- Examples include EELS, micro Raman imaging, force-distance curve maps in AFM, Scanning tunneling spectroscopy, etc.
- Usually, the 3D matrix is converted to a 2D matrix
 $M \text{ (rows)} \times N \text{ (columns)} \times P \text{ (spectral points)}$

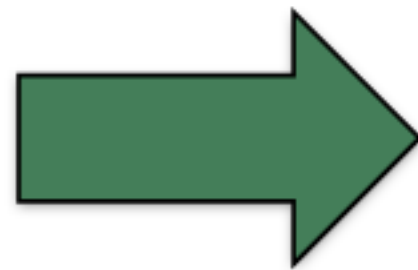
$$= (MN) \times P$$

Pixels x spectral points



Matrix Factorization

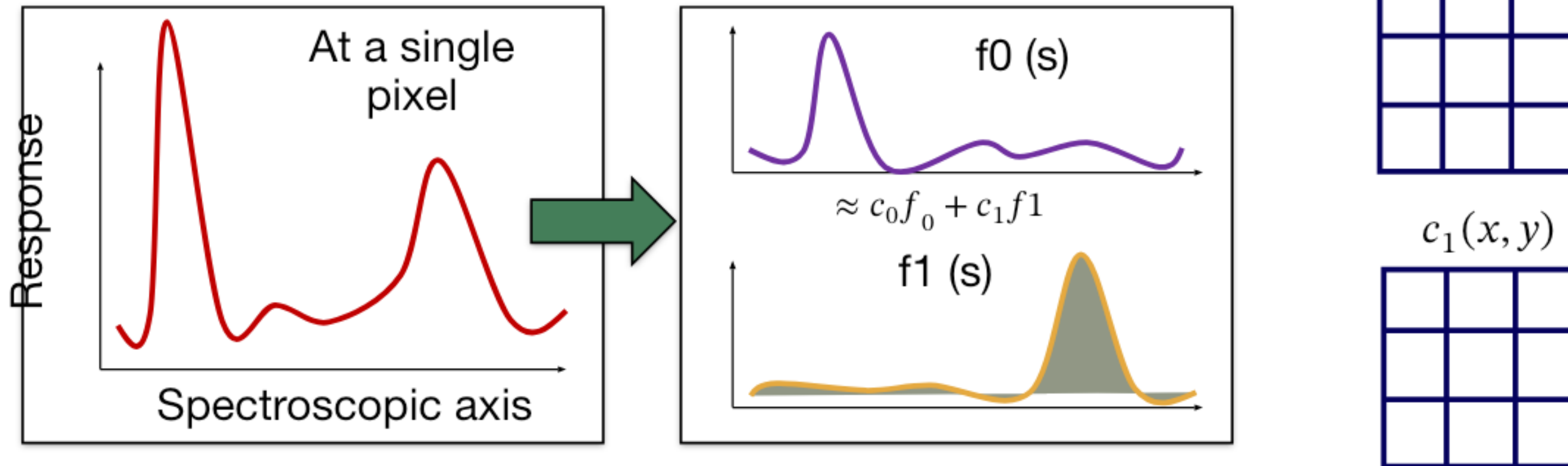
- We now have a 2D matrix (call it **X**) of size (MN,P)
- If **X** is 'small' (e.g., (10,10)) we can comprehend it easily. But if the size of X is (2500, 32), then this becomes more difficult
- The number of spectral points **P** is often termed the "number of dimensions".
- To see trends in our data, we want to visualize them in some manner. But the 3D data cube is big, so what can we do?



Matrix Factorization

Dimensionality Reduction

- The reason it is difficult to visualize is because there are too many spectral points P per pixel. What we need is to reduce the dimensionality, while preserving most of the data structure.
- Alternative viewpoint: Represent the spectrum at pixel z by a linear expansion



(1) Singular Value Decomposition

$$X = USVT$$

Note: In the full decomposition, $r = n$.
Practically, $r \ll n$

X is of size (m,n)
 U is of size (m,r)
 S is of size (r,r)
 V is of size (n,r)

The columns of U, V
form the basis
 S scales them, in
descending order

By the SVD theorem, it is always possible to decompose real matrix X into $USVT$ where

U, S, V : **unique**

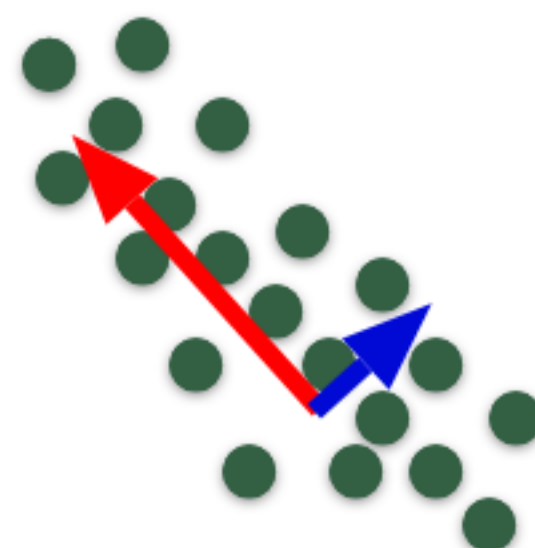
U, V : **column orthonormal**

- $UTU = I, VTV = I$
- Columns are orthogonal unit vectors

S : **diagonal**

- Positive, sorted in descending order

Finding the main
'directions' in the data



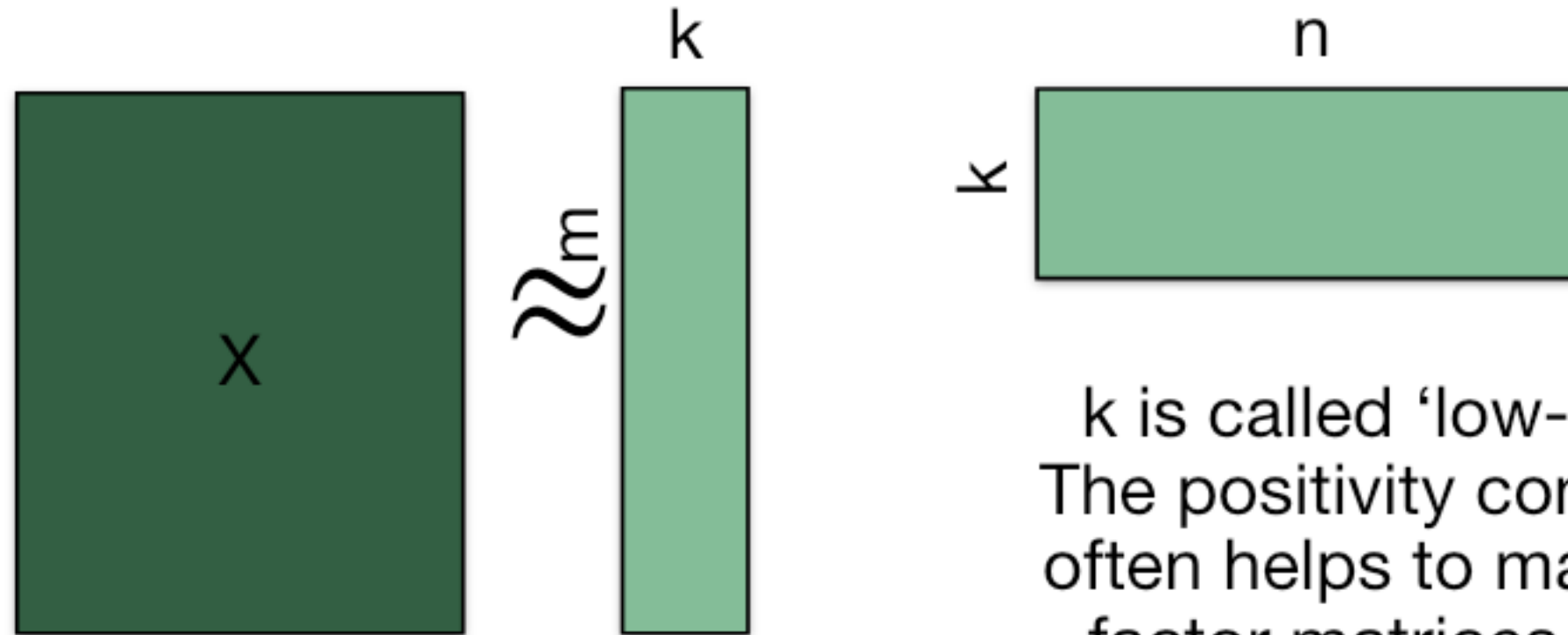
Non-negative Matrix Factorization

- SVD is an extremely powerful tool to visualize data and observe trends, but it has weaknesses. Mainly, it often produces highly unphysical behavior, for instance negative intensity values in spectra where only positive values are physically realizable
- Therefore, we can turn to other methods, such as non-negative matrix factorization approaches

Matrix Form

$$\mathbf{X} \approx \mathbf{WH}$$

Here, we enforce that $\mathbf{X}$, $\mathbf{W}$ and $\mathbf{H}$ are all non-negative matrices



k is called 'low-rank'. The positivity constraint often helps to make the factor matrices more interpretable.

Spectral Unmixing: N-FINDR

- Spectra for a given pixel is assumed to be a linear combination of the end-member spectra (+ Gaussian noise). The mixing proportions sum to 1

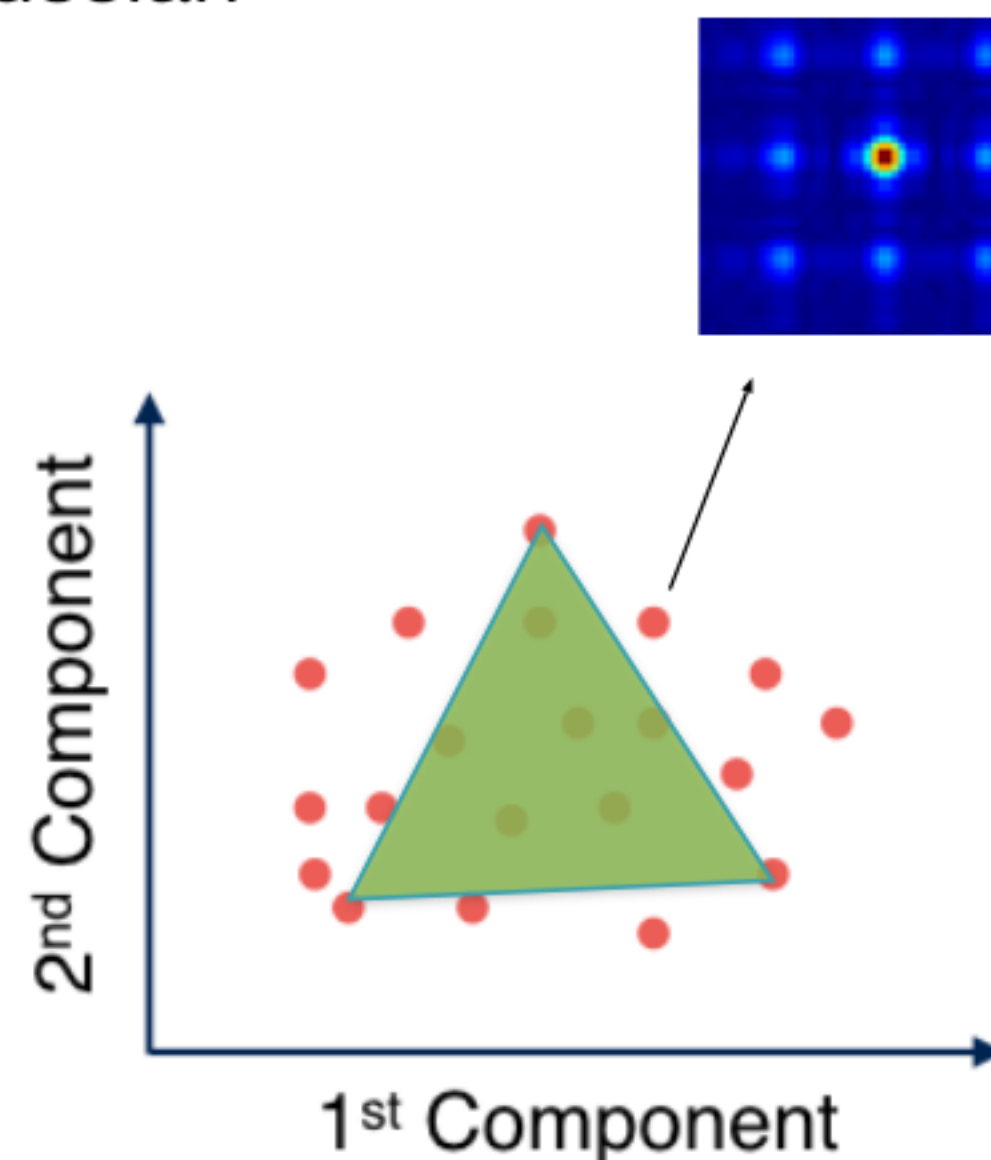
Physics constraint

$$p_{ij} = \sum_k e_{ik} c_{kj} + \varepsilon \quad \sum_k c_{kj} = 1$$

- Let E be the matrix of end-members (here, 3).

$$E = \begin{bmatrix} \vec{1} & \vec{1} & \vec{1} \\ e_1 & e_2 & e_3 \end{bmatrix} \quad V \left(\frac{1}{(l-1)!} \right) |\det(E)|$$

- Iteratively select endmembers, accepting the new selection if the volume increases



Segmenting based on phases

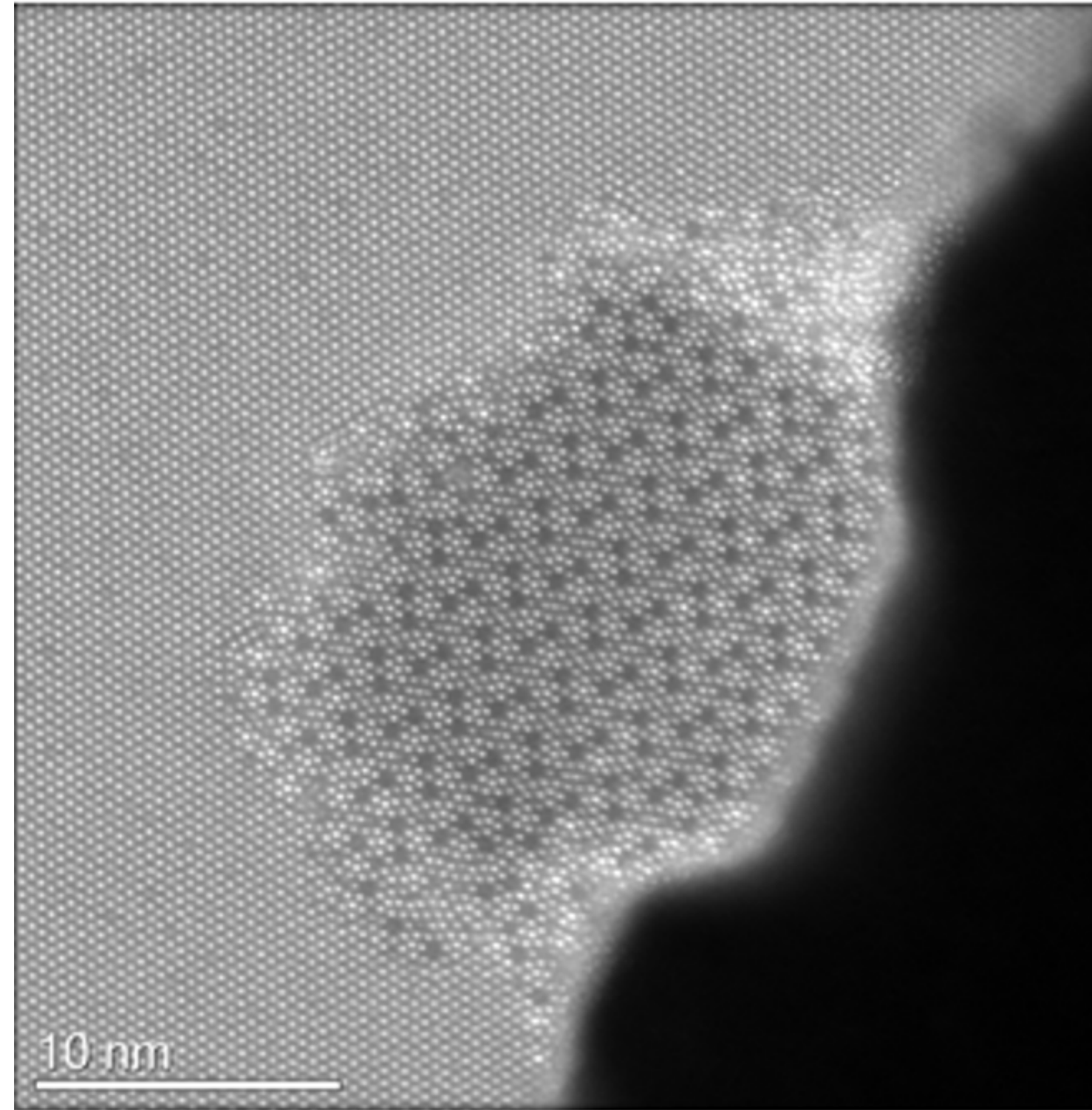
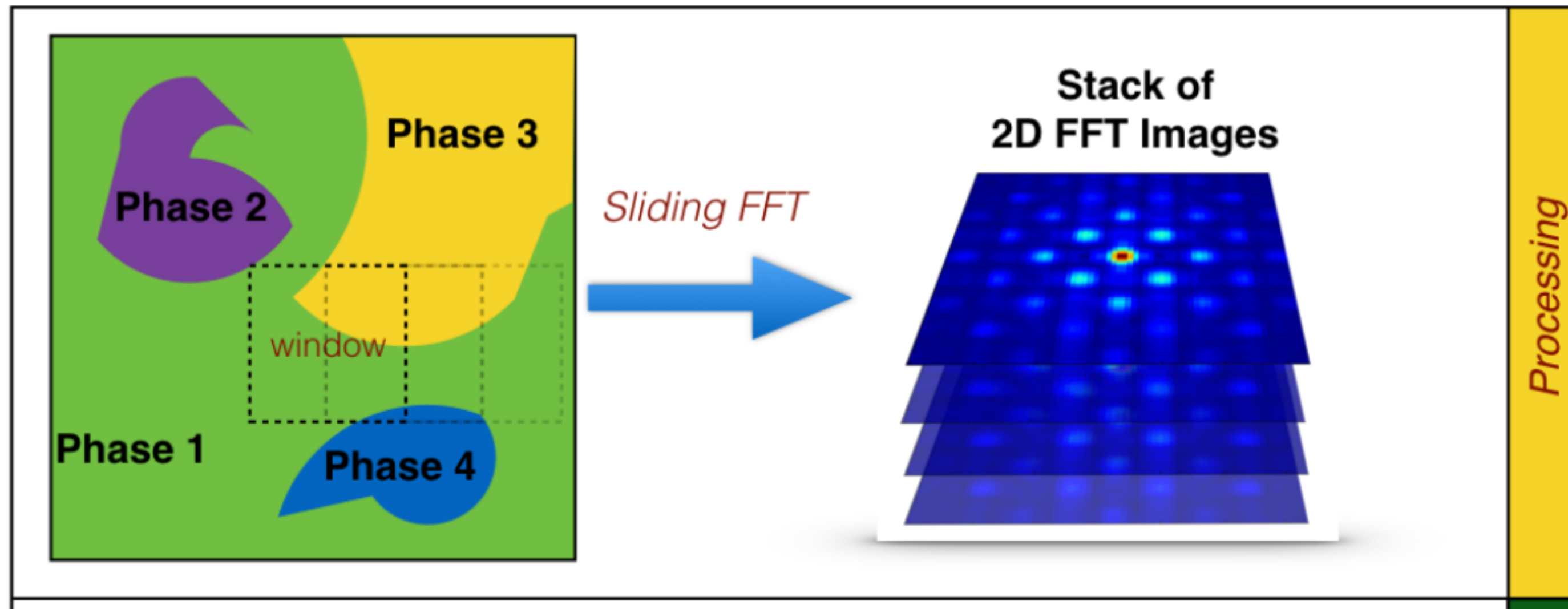


Image courtesy of
A. Borisevich (ORNL)
and
Q. He
(ORNL/Manchester)

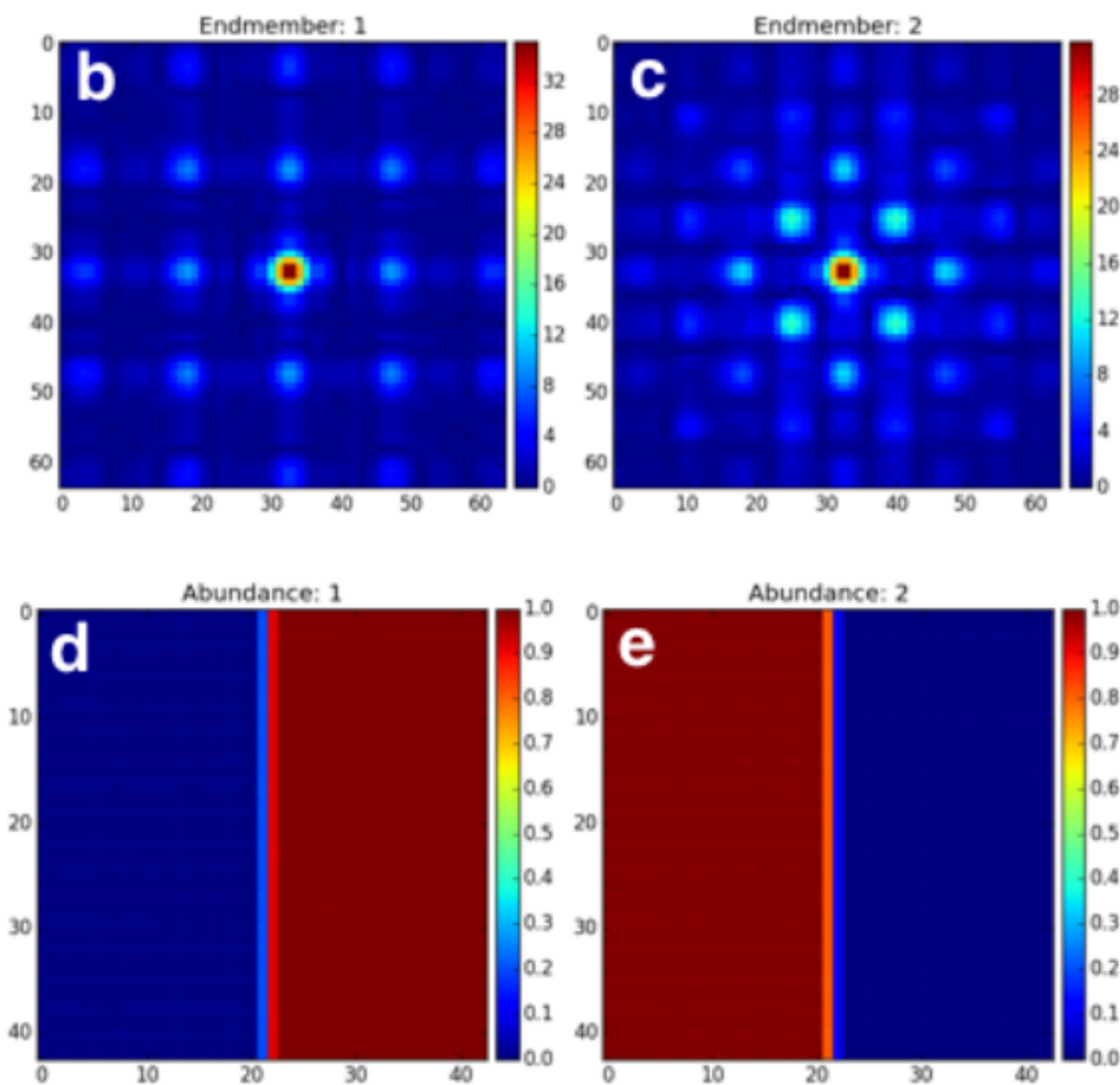
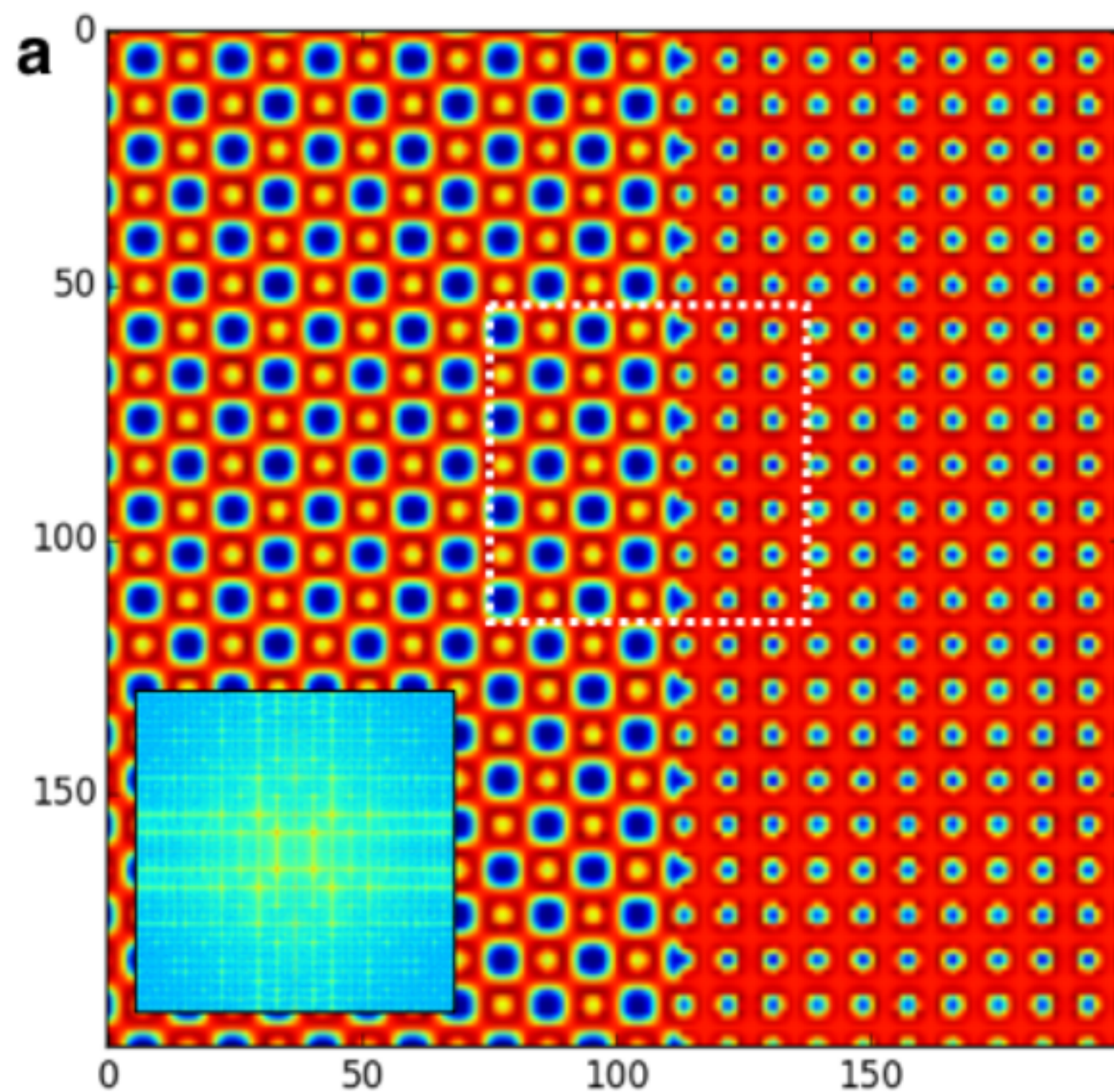
How can we segment this image based on the
phases present?

Sliding Window Method



Now, we can try to do spectral unmixing on the 2D stack of FFT images

Ideal test case



An LLM is a next-token model wrapped in context

What the model learns

Patterns in language, code, tables, figures, and scientific text from training data. The trained weights are fixed during an ordinary chat.

What the prompt supplies

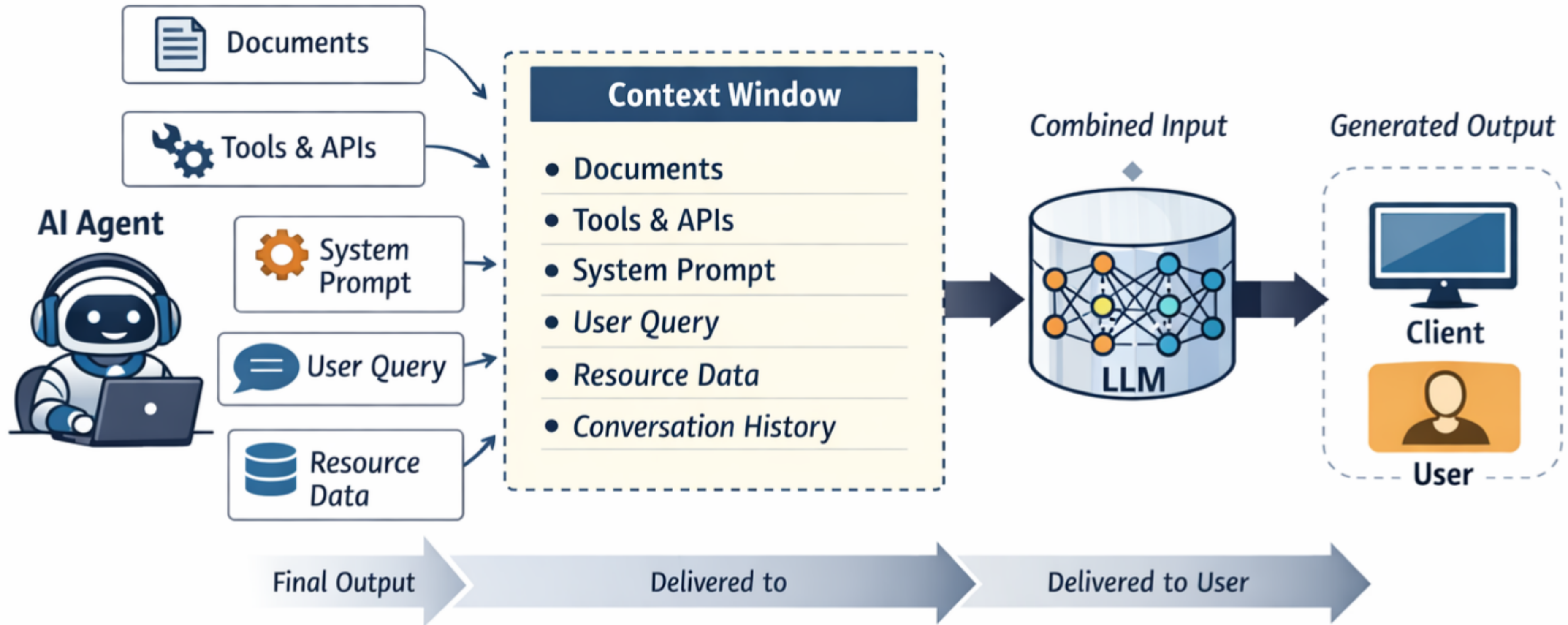
The current task, conversation history, retrieved documents, notebook state, and tool results. This is the working memory for the next response.

What makes it an agent

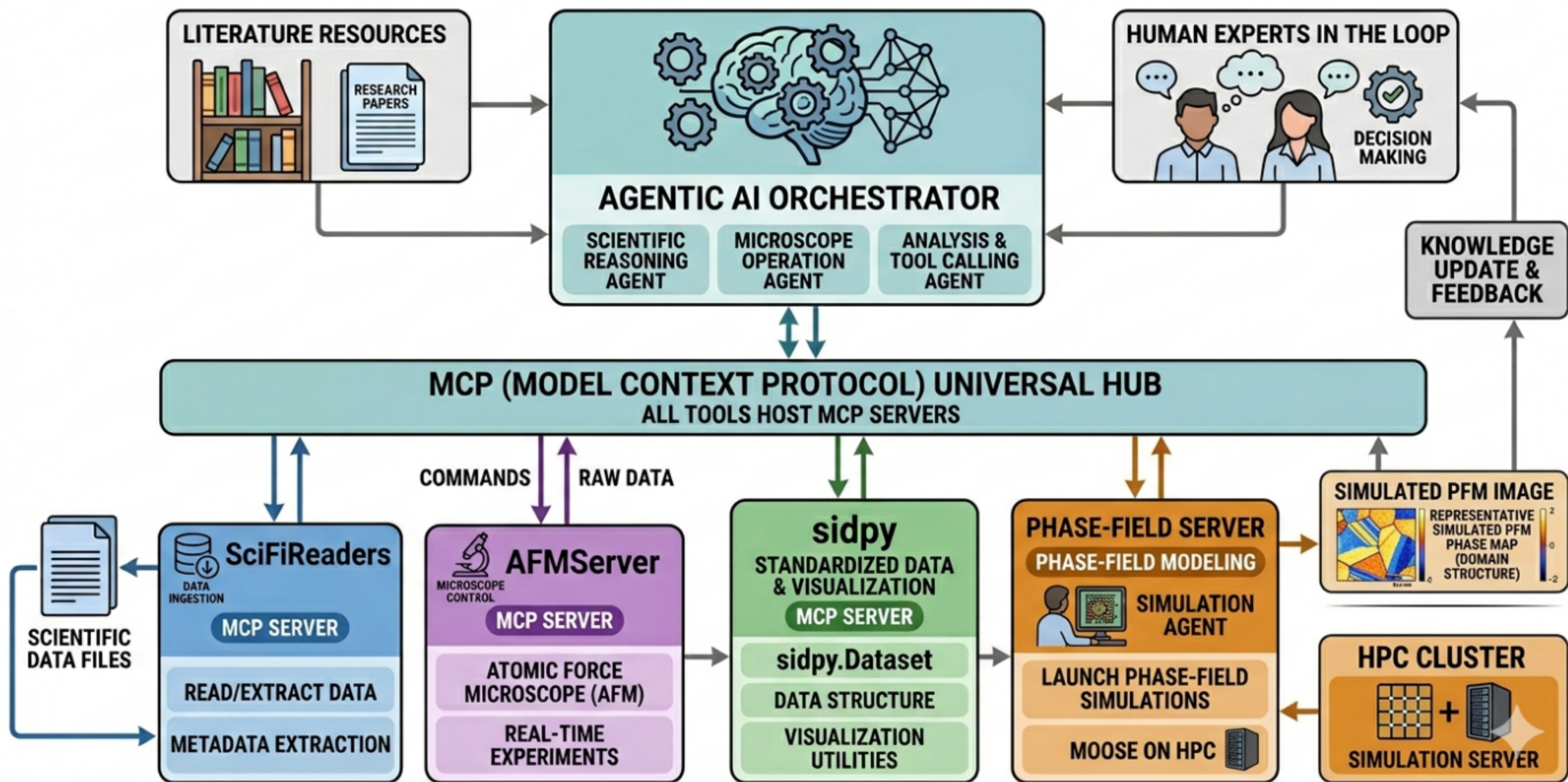
A loop that lets the LLM choose actions, call tools, observe results, and update the next step. In microscopy, those tools can include simulators, analysis code, and instrument APIs.



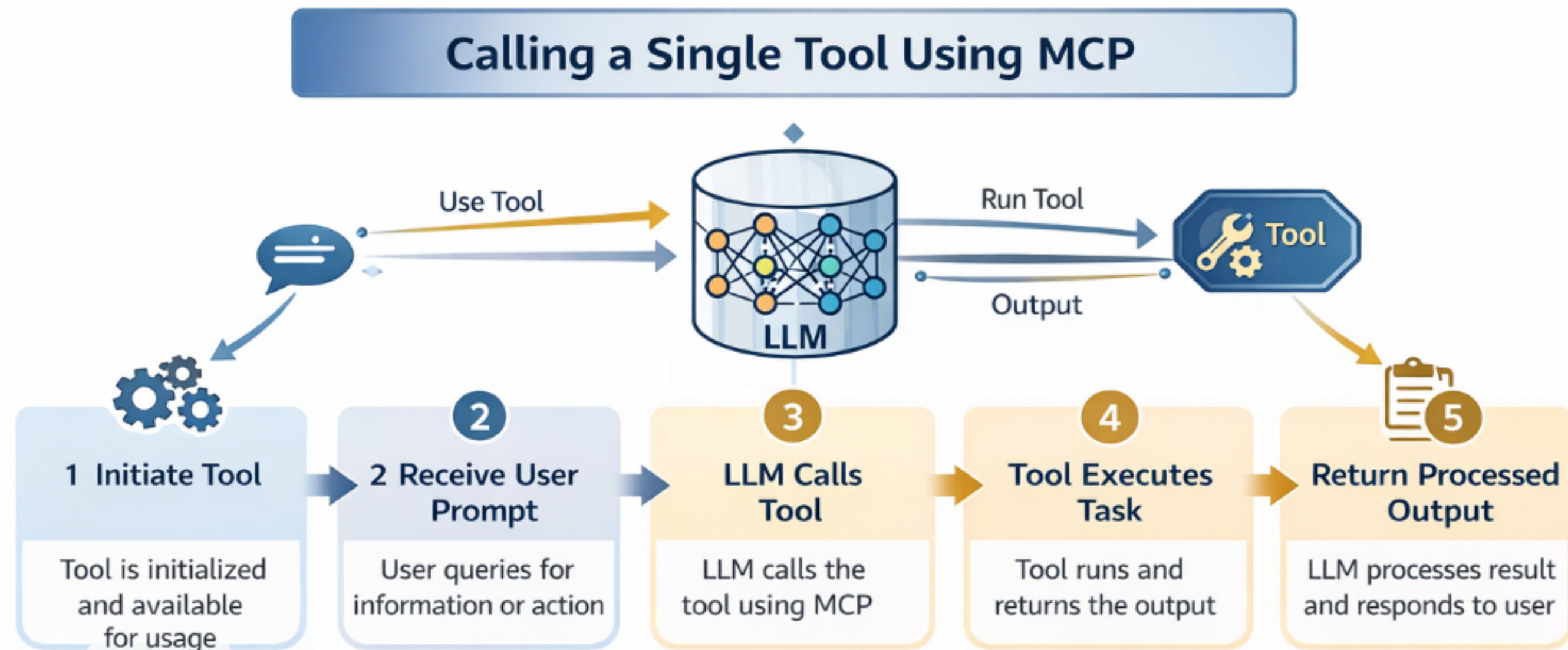
LLMs are stateless models



AI-DRIVEN MICROSCOPY WORKFLOW UTILIZING MCP SERVER ECOSYSTEM



MCP lets an LLM call tools, observe results, and continue



Specifics of MCP Tool Usage

Calling a Single Tool Using MCP

